

VALUE ADDED COURSE ON AI/ ML - TASK 4

NAME: THARUN VINAYAK B

USN: 24BBCAG039

COURSE: BCA (GENERAL)

SEMESTER: 5TH / YEAR: III

SUBJECT: ARTIFICIAL INTELLIGENCE

EXPERIMENT 4: Product Review Sentiment Analyzer using Natural Language Processing

AIM:

To analyze product review sentiment using **VADER and TF-IDF with Logistic Regression**, and evaluate their performance using **Accuracy, Precision, Recall, F1-Score, and Confusion Matrix**.

SYSTEM REQUIREMENTS:

Hardware Requirements

- Processor: Intel Core i3 or equivalent (minimum)
- RAM: 4 GB or higher
- Internet Connection: Required (for downloading NLTK lexicons and raw dataset in Google Colab)

Software Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Platform: Google Colab / Jupyter Notebook
- Environment: Python 3.x
- Required Libraries: pandas, numpy, nltk, scikit-learn

PROCEDURE:

1. **Dataset Acquisition:** Load the benchmark product review dataset from a public URL.
2. **Text Preprocessing:** Clean the review text by removing HTML tags, punctuation, and special characters, and convert it to lowercase.
3. **VADER Sentiment Scoring:** Use NLTK VADER to calculate sentiment scores and classify reviews as positive or negative.
4. **Feature Extraction:** Convert cleaned reviews into **TF-IDF feature vectors**.
5. **Train-Test Split:** Split the data into **80% training and 20% testing** sets.
6. **Model Training & Evaluation:** Train a **Logistic Regression** classifier and evaluate its performance using **Accuracy, Precision, Recall, F1-Score, and Confusion Matrix**.
7. **Real-Time Inference:** Test VADER and Logistic Regression on custom product reviews.

B. THARUN VINAYAK
24BBCAG039

PROGRAM:

```
# ============================================================================
# Experiment 4: Product Review Sentiment Analyzer
# Dataset: IMDb Product/Review Sentiment Dataset
# ============================================================================

import pandas as pd
import re
import nltk

from nltk.sentiment.vader import SentimentIntensityAnalyzer

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Download VADER Lexicon
nltk.download('vader_lexicon', quiet=True)

# Step 1: Load Dataset
DATA_URL = "https://raw.githubusercontent.com/Ankit152/IMDB-sentiment-
analysis/refs/heads/master/IMDB-Dataset.csv"

df = pd.read_csv(DATA_URL).head(3000)

print("--- Step 1: Dataset Overview ---")
print(f"Total Review Samples: {df.shape[0]}")
print("\n")
print(df.head())
print("\n" + "=" * 60 + "\n")



# Step 2: Clean Text Data
def clean_text(text):
    text = re.sub(r'<.*?>', '', text)          # Remove HTML tags
    text = re.sub(r'[^a-zA-Z\s]', '', text)    # Remove special characters
    text = text.lower().strip()                # Convert to lowercase
    return text
```


B. THARUN VINAYAK
24BBCAG039

```
df['clean_review'] = df['review'].apply(clean_text)

# Convert sentiment into numbers
df['target'] = df['sentiment'].map({
    'positive': 1,
    'negative': 0
})



# Step 3: VADER Sentiment Analysis
sia = SentimentIntensityAnalyzer()

def get_vader_sentiment(text):
    score = sia.polarity_scores(text)['compound']
    return 1 if score >= 0.05 else 0



df['vader_pred'] = df['review'].apply(get_vader_sentiment)

vader_acc = accuracy_score(
    df['target'],
    df['vader_pred']
)

print("--- Step 2: VADER Performance ---")
print(f"VADER Accuracy: {vader_acc * 100:.2f}%")
print("\n" + "=" * 60 + "\n")



# Step 4: TF-IDF Feature Extraction
X = df['clean_review']
y = df['target']

tfidf = TfidfVectorizer(
    max_features=5000,
    stop_words='english'
)

X_tfidf = tfidf.fit_transform(X)

print("--- Step 3: TF-IDF Feature Extraction ---")
```

```
print(f"TF-IDF Matrix Shape: {X_tfidf.shape}")
print("\n" + "=" * 60 + "\n")


# Step 5: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)


# Step 6: Logistic Regression Model
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)


# Step 7: Model Evaluation
y_pred = model.predict(X_test)

ml_acc = accuracy_score(y_test, y_pred)

print("--- Step 4: Logistic Regression Performance ---")
print(f"Accuracy: {ml_acc * 100:.2f}%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=['Negative', 'Positive']
    )
)

print("=" * 60)
```


B. THARUN VINAYAK
24BBCAG039

```
# Step 8: Real-Time Product Review Analysis
def analyze_product_review(review):

    cleaned = clean_text(review)

    # Logistic Regression prediction
    vector = tfidf.transform([cleaned])

    ml_prediction = model.predict(vector)[0]

    probability = model.predict_proba(vector)[0][ml_prediction]

    # VADER prediction
    vader_score = sia.polarity_scores(review)['compound']

    if ml_prediction == 1:
        sentiment = "POSITIVE"
    else:
        sentiment = "NEGATIVE"

    print("\nReview:", review)
    print("ML Prediction:", sentiment)
    print(f"Confidence: {probability * 100:.1f}%")
    print(f"VADER Compound Score: {vader_score:.3f}")


# Step 9: Test Custom Reviews
print("\n--- Step 5: Custom Product Reviews ---")

analyze_product_review(
    "The battery life is terrible and it overheats within 10 minutes. Waste of money!"
)

analyze_product_review(
    "Build quality feels premium, sleek design, and sound clarity exceeded my expectations."
)

print("\n" + "=" * 60)
```

OUTPUT:

```
--- Step 1: Dataset Overview ---
Total Review Samples: 3000


                                              review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive

============================================================

--- Step 2: VADER Performance ---
VADER Accuracy: 70.63%

============================================================

--- Step 3: TF-IDF Feature Extraction ---
TF-IDF Matrix Shape: (3000, 5000)

============================================================

--- Step 4: Logistic Regression Performance ---
Accuracy: 83.17%

Confusion Matrix:
[[240  58]
 [ 43 259]]

Classification Report:
              precision    recall  f1-score   support

    Negative       0.85      0.81      0.83       298
    Positive       0.82      0.86      0.84       302

    accuracy                           0.83       600
   macro avg       0.83      0.83      0.83       600
weighted avg       0.83      0.83      0.83       600

============================================================

--- Step 5: Custom Product Reviews ---

Review: The battery life is terrible and it overheats within 10 minutes. Waste of money!
ML Prediction: NEGATIVE
Confidence: 95.5%
VADER Compound Score: -0.735

Review: Build quality feels premium, sleek design, and sound clarity exceeded my expectations.
ML Prediction: POSITIVE
Confidence: 60.9%
VADER Compound Score: 0.402

============================================================
```


B. THARUN VINAYAK
24BBCAG039

RESULT:

The Product Review Sentiment Analyzer successfully analyzed IMDb reviews using **VADER** and **TF-IDF with Logistic Regression**. The model classified reviews as positive or negative and displayed the **accuracy, confusion matrix, classification report, confidence score, and VADER compound score** for custom product reviews.

B. THARUN VINAYAK
24BBCAG039